

Проект. Неравновесная агрегация, фрактальные кластеры

Этап 3

НФИбд-01-22: Ощепков Дмитрий, Хамдамова Айжана, Козлов В

Содержание

1	Вводная часть	5
1.1	Актуальность	5
2	Цель работы	6
2.1	Объект и предмет исследования	6
2.2	Материалы и методы	6
3	Теоретическое описание задачи	7
3.1	Фрактальная размерность	7
4	Агрегация, ограниченная диффузией	8
5	Практическая реализация	9
5.1	Описание алгоритма	9
5.2	Случайное блуждание	9
5.3	DLA кластер(Diffusion-Limited Aggregation)	10
5.4	DLA кластер(Diffusion-Limited Aggregation)	14
5.5	BA(Ballistic Aggregation)	15
5.6	BA(Ballistic Aggregation)	18
5.7	CLA кластер	18
5.8	CLA кластер(Chemically Limited Aggregationcluster)	20
5.9	CCA(Cluster-Cluster Aggregation)	20
5.10	CCA(Cluster-Cluster Aggregation)	24
5.11	Фрактальная размерность	24
5.12	CLA кластер	25
5.13	CCA (Cluster-Cluster Aggregation)	25
6	Объединение кластеров	26
7	Выводы	27
8	Список литературы	28

Список иллюстраций

5.1	Блок-схема алгоритма модели DLA	9
5.2	DLA кластер	14
5.3	DLA кластер	15
5.4	BA кластер	18
5.5	CLA кластер	20
5.6	ССА кластер	24
5.7	График зависимости числа частиц в кластере от радиуса гирации	25

Список таблиц

1 Вводная часть

1.1 Актуальность

Существуют разнообразные физические процессы, основная черта которых — неравновесная агрегация:

- образование частиц сажи
- выращивание кристаллов соли
- распространение воды в нефти

2 Цель работы

Исследовать модель агрегации, ограниченной диффузией(DLA).

Задачи

- Построить модель агрегации, ограниченной диффузией
- Найти размерность, получившихся кластеров
- Построить график зависимости числа частиц в кластере от радиуса агирации

2.1 Объект и предмет исследования

- Модель DLA, BA, CLA, CCA
- Фрактальная размерность
- График зависимости числа частиц в кластере от радиуса агирации

2.2 Материалы и методы

- Язык программирования Julia

3 Теоретическое описание задачи

3.1 Фрактальная размерность

$$d = \lim_{\epsilon \rightarrow 0} \left(\frac{\ln(N(\epsilon))}{\ln(\frac{1}{\epsilon})} \right)$$

$$\ln(N(\epsilon)) = D \ln(R) + b,$$

где D – фрактальная размерность, $N(\epsilon)$ – число частиц на расстоянии меньшем чем R , R – радиус.

4 Агрегация, ограниченная диффузией

Агрегация, ограниченная диффузией (diffusion-limited aggregation, DLA) — первая модель агрегации, представляющая собой шумный рост, ограниченный диффузией.

5 Практическая реализация

5.1 Описание алгоритма

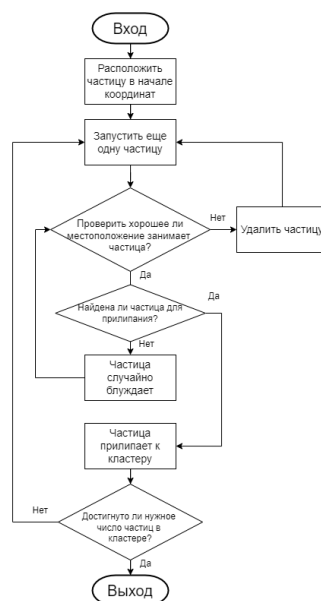


Рис. 5.1: Блок-схема алгоритма модели DLA

5.2 Случайное блуждание

Обозначим $v^u = (0, 1)$, $v^d = (0, -1)$, $v^r = (1, 0)$, $v^l = (-1, 0)$ - шаг на 1 вверх, вниз, влево, вправо соответственно.

$\{S_n\}$ – ряд, описывающий случайное блуждание, $*$ = u, d, r, l , n – количество шагов

$$S_n = \sum_{i=1}^n v_n^*,$$

$$P(v_{i+1} = v_n^*) = \frac{1}{4}$$

5.3 DLA кластер(Diffusion-Limited Aggregation)

```

using Plots
using Random
using ColorSchemes

# -----
# Diffusion-Limited Aggregation (DLA)
# -----

mutable struct DLACluster
    size::Int
    target::Int
    grid::Matrix{Int}
    particles::Vector{Tuple{Int,Int}}
end

function DLACluster(size=100, target=500)
    grid = zeros{Int, size, size}
    mid = size ÷ 2
    grid[mid, mid] = 1
    DLACluster(size, target, grid, [(mid, mid)])
end

```

```

function simulate!(dla::DLACluster)
    directions = [(1,0), (-1,0), (0,1), (0,-1)]
    while length(dla.particles) < dla.target
        edge = rand(["top", "bottom", "left", "right"])
        x, y = start_position(dla.size, edge)

        while true
            dx, dy = rand(directions)
            x += dx
            y += dy

            if !check_bounds(dla.size, x, y)
                break
            end

            if check_neighbors(dla.grid, x, y)
                dla.grid[x, y] = 1
                push!(dla.particles, (x, y))
                break
            end
        end
    end
end

function plot_dla(dla::DLACluster; filename="DLA_julia.png")
    x = [p[1] for p in dla.particles]
    y = [p[2] for p in dla.particles]

    plt = scatter(y, x,

```

```

        markersize=1,
        color=:viridis,
        legend=false,
        axis=false,
        grid=false,
        title="DLA Cluster (Julia)",
        aspect_ratio=1
    )
    savefig(plt, filename)
end

# Вспомогательные функции
function start_position(size, edge)
    if edge == "top"
        (rand(1:size), 1)
    elseif edge == "bottom"
        (rand(1:size), size)
    elseif edge == "left"
        (1, rand(1:size))
    else
        (size, rand(1:size))
    end
end

end

check_bounds(size, x, y) = 1 ≤ x ≤ size && 1 ≤ y ≤ size

function check_neighbors(grid, x, y)
    for dx in -1:1, dy in -1:1
        (dx == 0 && dy == 0) && continue
    end
end

```

```

        nx, ny = x + dx, y + dy
        1 ≤ nx ≤ size(grid, 1) && 1 ≤ ny ≤ size(grid, 2) && grid[nx, ny] == 1 && return true
    end
    false
end

# Заныск
Random.seed!(42)
dla = DLACluster(150, 3000)
simulate!(dla)
plot_dla(dla)

```

5.4 DLA кластер(Diffusion-Limited Aggregation)

DLA Cluster (3000 particles)

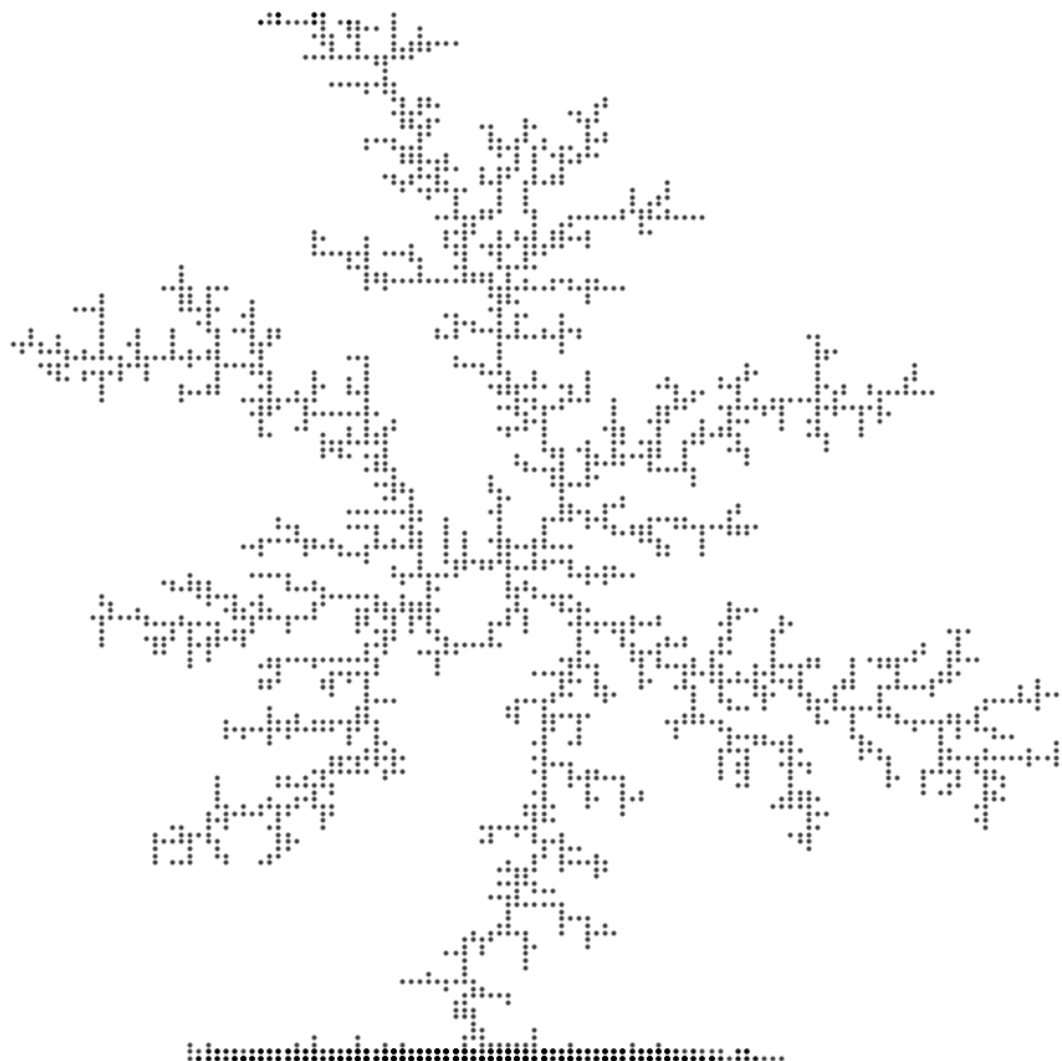


Рис. 5.2: DLA кластер

DLA Cluster (5000 particles)

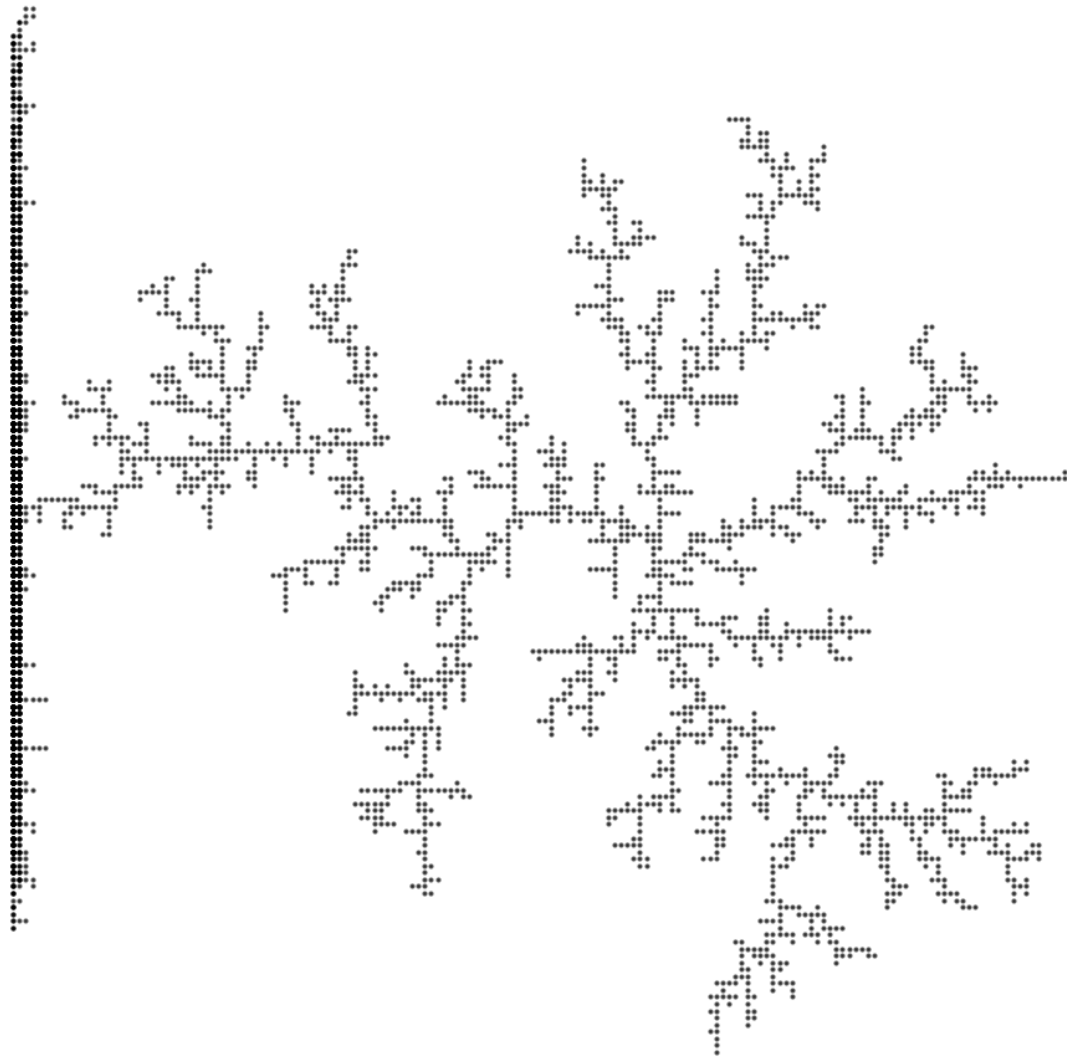


Рис. 5.3: DLA кластер

5.5 BA(Ballistic Aggregation)

`using Plots`

`using Random`

`using ColorSchemes`

```

## =====
## Ballistic Aggregation (BA)
## =====

mutable struct BACluster
    size::Int
    target::Int
    grid::Matrix{Int}
    particles::Vector{Tuple{Int,Int}}
end

function BACluster(size=100, target=500)
    grid = zeros{Int, size, size}
    mid = size ÷ 2
    grid[mid, mid] = 1
    BACluster(size, target, grid, [(mid, mid)])
end

function simulate!(ba::BACluster)
    while length(ba.particles) < ba.target
        x, y = rand_start_position(ba.size)
        prev = (x, y)

        while check_bounds(ba.size, x, y)
            dx = sign(ba.size÷2 - x)
            dy = sign(ba.size÷2 - y)
            x += dx
            y += dy
        end
    end

```



```

        if check_neighbors(ba.grid, x, y)
            ba.grid[prev...] = 1
            push!(ba.particles, prev)
            break
        end
        prev = (x, y)
    end
end

end

function plot_ba(ba::BACluster; filename="BA_julia.png")
    x = [p[1] for p in ba.particles]
    y = [p[2] for p in ba.particles]

    plt = scatter(y, x,
        markersize = 1.5,
        color = :blues,
        legend = false,
        axis = false,
        title = "Ballistic Aggregation",
        aspect_ratio = 1
    )
    savefig(plt, filename)
end

```

5.6 BA(Ballistic Aggregation)

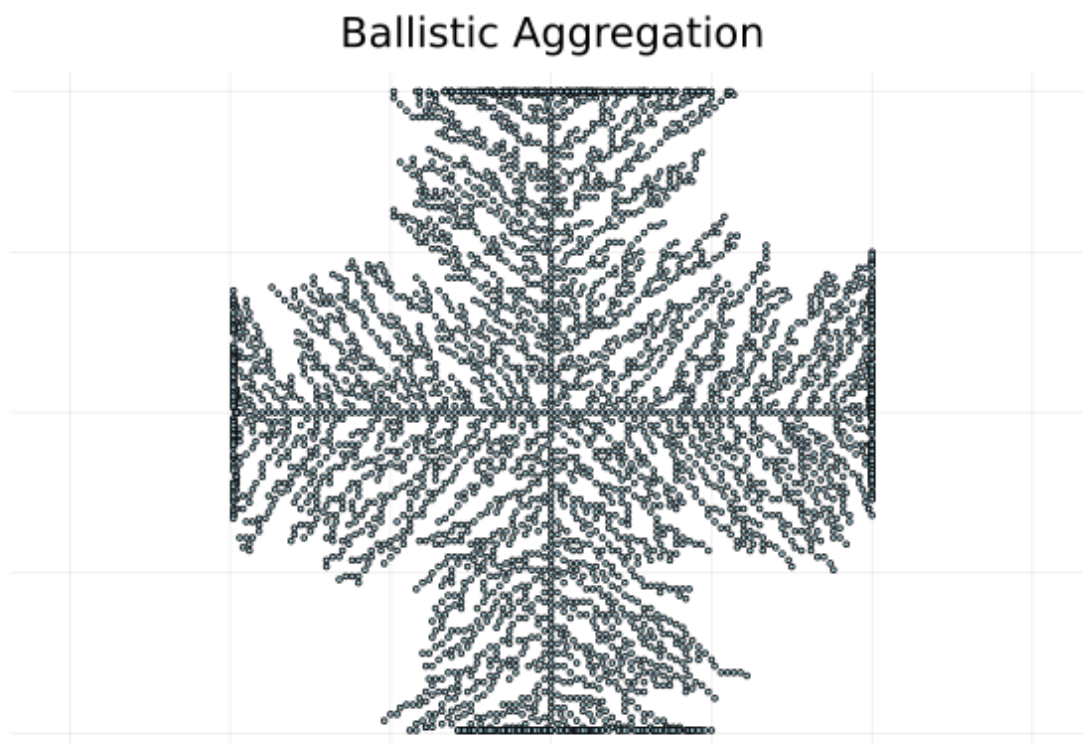


Рис. 5.4: ВА кластер

5.7 CLA кластер

```
mutable struct CLACluster
    size::Int
    target::Int
    grid::Matrix{Int}
    particles::Vector{Tuple{Int,Int}}
end

function CLACluster(size=100, target=500)
    grid = zeros{Int, size, size}
```

```

mid = size ÷ 2
grid[mid, mid] = 1
CLACluster(size, target, grid, [(mid, mid)])
end

function simulate!(cla::CLACluster)
    directions = [(-1,0), (1,0), (0,-1), (0,1)]

    while length(cla.particles) < cla.target
        # Старт с случайной позиции на границе
        edge = rand(["top", "bottom", "left", "right"])
        x, y = start_position(cla.size, edge)

        while true
            dx, dy = rand(directions)
            x += dx
            y += dy

            if !check_bounds(cla.size, x, y)
                break
            end

            if check_neighbors(cla.grid, x, y)
                cla.grid[x, y] = 1
                push!(cla.particles, (x, y))
                break
            end
        end
    end
end
end

```

end

5.8 CLA кластер(Chemically Limited Aggregationcluster)

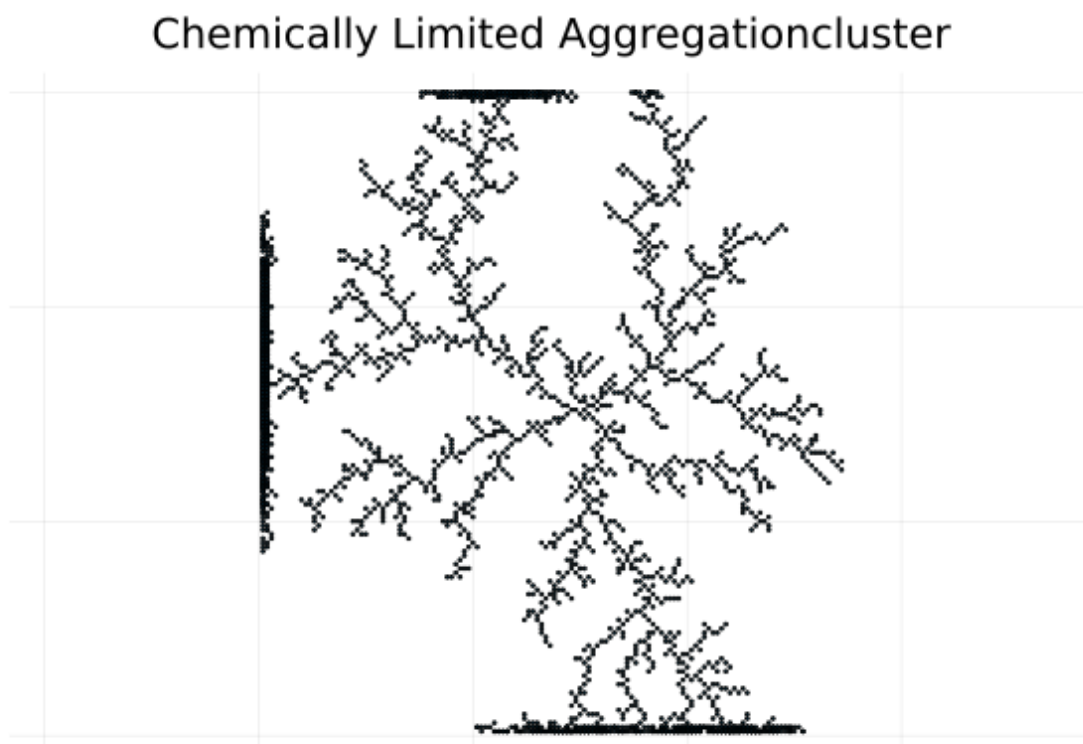


Рис. 5.5: CLA кластер

5.9 CCA(Cluster-Cluster Aggregation)

```
## =====  
## Cluster-Cluster Aggregation (CCA)  
## =====  
mutable struct ClusterCCA  
    particles::Vector{Tuple{Int,Int}}  
    position::Tuple{Float64,Float64}
```

```

        velocity::Tuple{Float64,Float64}
    end

    struct CCA
        size::Int
        clusters::Vector{ClusterCCA}
    end

    function CCA(size=100, n_clusters=20)
        clusters = [ClusterCCA(
            [(rand(1:size), rand(1:size))],
            (rand(1:size), rand(1:size)),
            (randn(), randn())
        ) for _ in 1:n_clusters]
        CCA(size, clusters)
    end

    function simulate!(cca::CCA, steps=100)
        for _ in 1:steps
            move_clusters!(cca)
            merge_clusters!(cca)
        end
    end

    function move_clusters!(cca::CCA)
        for cluster in cca.clusters
            x, y = cluster.position
            vx, vy = cluster.velocity
            x = mod(x + vx, cca.size)

```

```

        y = mod(y + vy, cca.size)
        cluster.position = (x, y)
    end
end

function merge_clusters!(cca::CCA)
    merged = Set{Int}()
    new_clusters = ClusterCCA[]

    for (i, c1) in enumerate(cca.clusters)
        i in merged && continue
        for (j, c2) in enumerate(cca.clusters[i+1:end])
            dist = hypot(c1.position[1]-c2.position[1], c1.position[2]-c2.position[2])
            if dist < 3.0
                push!(merged, i)
                push!(merged, i+j)
                new_particles = vcat(c1.particles, c2.particles)
                push!(new_clusters, ClusterCCA(
                    new_particles,
                    ((c1.position[1]+c2.position[1])/2, (c1.position[2]+c2.position[2])
                    ((c1.velocity[1]+c2.velocity[1])/2, (c1.velocity[2]+c2.velocity[2])
                ))
            end
        end
        i ∉ merged && push!(new_clusters, c1)
    end
    cca.clusters = new_clusters
end

```

```

function plot_cca(cca::CCA; filename="CCA_julia.png")
    plt = plot(legend=false, axis=false, aspect_ratio=1)
    colors = ColorSchemes.rainbow

    for (i, cluster) in enumerate(cca.clusters)
        x = [p[1] for p in cluster.particles]
        y = [p[2] for p in cluster.particles]
        scatter!(plt, y, x,
            markersize=1.2,
            color=colors[(i % 10)+1]
        )
    end
    savefig(plt, filename)
end

```

5.10 CCA(Cluster-Cluster Aggregation)

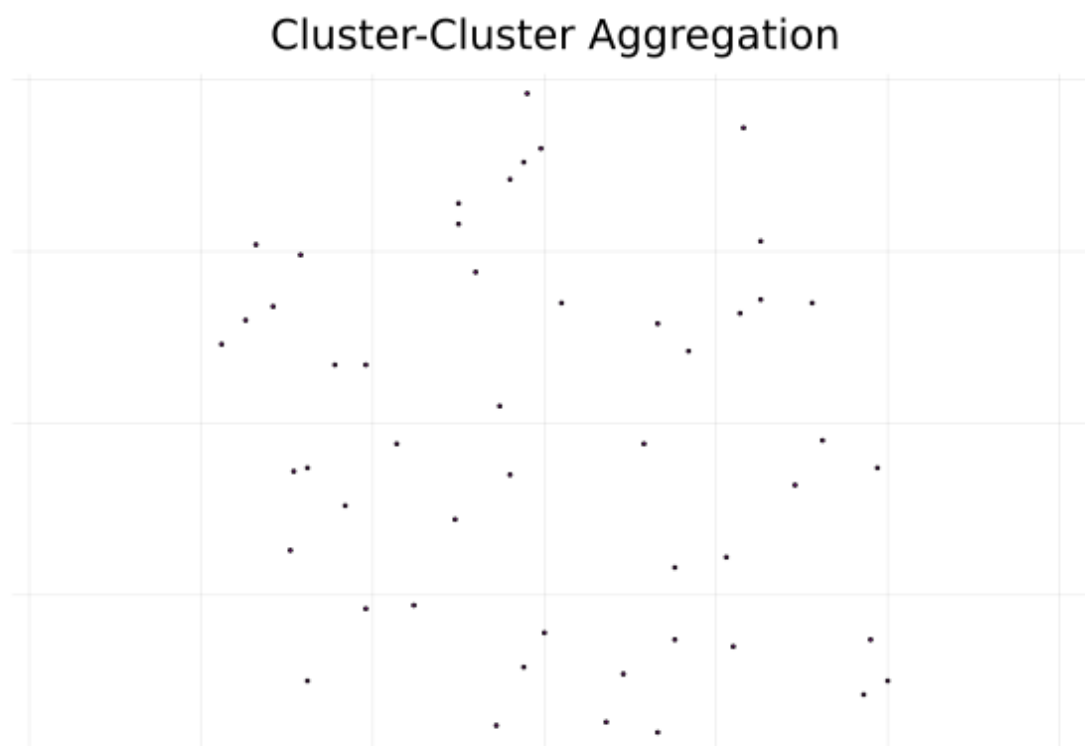


Рис. 5.6: CCA кластер

5.11 Фрактальная размерность

При построении 17 моделей с ограничением по радиусу от 130 до 290 получили $D = 1.717$.

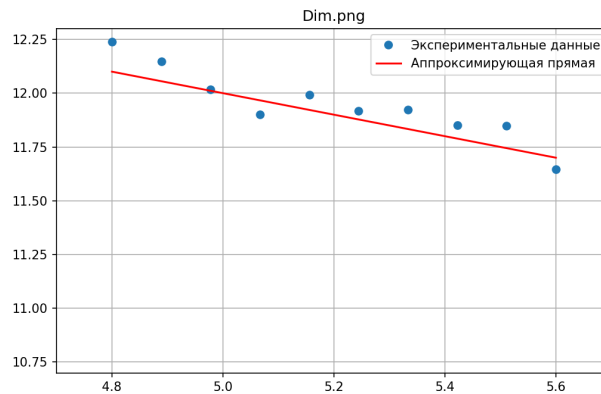


Рис. 5.7: График зависимости числа частиц в кластере от радиуса гирации

5.12 CLA кластер

5.13 CCA (Cluster-Cluster Aggregation)

Основные этапы алгоритма: 1. Инициализация кластеров со случайными позициями 2. Движение кластеров с постоянной скоростью 3. Объединение при сближении (дистанция < 3.0) 4. Визуализация с цветовой дифференциацией

Особенности реализации: - Динамическое обновление позиций (`mod()` для границ) - Рекурсивное объединение кластеров - Автоматическая цветовая палитра (Rainbow) - Оптимизированная обработка коллизий

```
“julia mutable struct ClusterCCA particles::Vector{Tuple{Int,Int}} position::Tuple{Float64,Float64}
velocity::Tuple{Float64,Float64} end

struct CCA size::Int clusters::Vector{ClusterCCA} end

function CCA(size=100, n=20) clusters = [ClusterCCA([(rand(1:size), rand(1:size))],
(rand(1:size), rand(1:size)), (randn(), randn())) for _ in 1:n] CCA(size, clusters) end

function simulate!(cca, steps=100) for _ in 1:steps for c in cca.clusters x,y = c.position
.+ c.velocity c.position = (mod(x, cca.size), mod(y, cca.size)) end merge_clusters!(cca)
end end
```

6 Объединение кластеров

```
function merge_clusters!(cca) merged = Set{Int}() new_clusters = ClusterCCA[] # ...  
(логика объединения) cca.clusters = new_clusters end
```

7 Выводы

- Построены модели DLA, BA, CLA, CCA
- Найдена фрактальная размерность, получившихся кластеров DLA
- Построен график зависимости числа частиц в кластере от радиуса гирации DLA

8 Список литературы

1. Медведев Д.А. и др. Моделирование физических процессов и явлений на ПК: Учеб. пособие. Новосибирск: Новосиб. гос. ун-т, 2010. 101 с.
2. Sander L.M. Diffusion-limited aggregation: A kinetic critical phenomenon? Contemporary Physics, 2000.